

Python Containers: Complete Training Notes

Lists, Tuples, Sets, Dictionaries, Comprehensions, Indexing, Slicing, Mutability, Methods, Real-World Use Cases and Interview Revision

Goal: These notes are written as future-reference training notes: first understand the concept, then see the syntax, code, output, real-world use case, common mistake, and interview point.

1. Python Containers — Big Picture

A **container** is a Python object used to store and organize multiple values. Different containers are designed for different requirements. Choosing the correct container is important because it affects readability, duplicate handling, ordering, and whether the data can be modified.

Type	Syntax	Ordered?	Mutable?	Duplicates?	Best use
List	[]	Yes	Yes	Yes	Changing collection of items
Tuple	()	Yes	No	Yes	Fixed collection of items
Set	{ } / set()	Do not rely on position	Yes	No	Unique values and set operations
Dictionary	{key: value}	Insertion order preserved	Yes	Key: No, Value: Yes	Indexed/key-value data

Important correction from the spoken class notes: A Python list is **mutable**, not immutable. A tuple and string are immutable. Also, sets are unordered collections and do **not** support normal indexing or slicing. Do not rely on which element set.pop() removes.

2. LIST

2.1 What is a List?

A **list** is an ordered and mutable Python container. It can store multiple values, including values of different data types. This is called a **heterogeneous** collection.

```
# A list can contain different data types
my_list = [10, "Python", 3.14, True]

print(my_list)
# Output:
# [10, 'Python', 3.14, True]
```

Code explanation: Square brackets create the list. The four elements are an integer, string, float, and Boolean. Python allows all of them in the same list.

2.2 Indexing

Each list element has a position called an **index**. Positive indexing starts from **0**. Negative indexing starts from **-1** at the end.

```
numbers = [10, 20, 30, 40, 50]

print(numbers[0])    # First element
print(numbers[2])    # Third element
print(numbers[-1])   # Last element

# Output:
# 10
# 30
# 50
```

2.3 Slicing

Slicing extracts part of a list. The general format is `list[start:stop:step]`. The stop position is excluded.

```
numbers = [10, 20, 30, 40, 50, 60]

print(numbers[1:4])    # Index 1 up to, but not including, index 4
print(numbers[:2])     # Every second element
print(numbers[::-1])   # Reverse the list

# Output:
# [20, 30, 40]
# [10, 30, 50]
# [60, 50, 40, 30, 20, 10]
```

2.4 Mutability

Mutable means the object can be changed after creation. Lists are mutable.

```
values = [1, 2, 3]

# Change the element at index 1
values[1] = 4

print(values)
# Output:
# [1, 4, 3]
```

The original list itself changed. This is one of the major differences between a list and a tuple.

2.5 Concatenation and Repetition

```
list1 = [1, 2, 3]
list2 = [4, 5]

combined = list1 + list2
repeated = list2 * 3

print(combined)
print(repeated)

# Output:
# [1, 2, 3, 4, 5]
# [4, 5, 4, 5, 4, 5]
```

The + operator creates a combined list. The * operator repeats the elements. If you write list1 + list2 without assigning the result, the original list is not permanently changed.

2.6 Important List Methods

append()

append() adds exactly one object to the end of a list.

```
numbers = [1, 2, 3]

numbers.append(4)

print(numbers)
# Output:
# [1, 2, 3, 4]
```

append() vs extend()

```
a = [1, 2]
a.append([3, 4])
print(a)
# [1, 2, [3, 4]]

b = [1, 2]
b.extend([3, 4])
print(b)
# [1, 2, 3, 4]
```

Difference: append() adds the entire object as one element. extend() takes elements from an iterable and adds them individually.

insert()

```
numbers = [10, 30]

# Insert 20 at index 1
numbers.insert(1, 20)

print(numbers)
# Output:
# [10, 20, 30]
```

pop()

pop() removes **and returns** an element. With no index, it removes the last element.

```
numbers = [10, 20, 30]

removed = numbers.pop()

print(removed) # 30
print(numbers) # [10, 20]
```

Because the removed item is returned, you can save it in a variable and use it later.

remove()

remove(value) removes the first matching value. It uses a **value**, not an index.

```
numbers = [10, 20, 20, 30]
```

```
numbers.remove(20)

print(numbers)
# Output:
# [10, 20, 30]
```

sort() and reverse()

```
numbers = [5, 2, 8, 1]

numbers.sort()
print(numbers)
# [1, 2, 5, 8]

numbers.reverse()
print(numbers)
# [8, 5, 2, 1]
```

Important: `sort()` and `reverse()` modify the original list. To sort descending directly, use `numbers.sort(reverse=True)`.

count() and index()

```
values = [10, 20, 20, 30]

print(values.count(20)) # Number of occurrences
print(values.index(30)) # Position of first occurrence

# Output:
# 2
# 3
```

2.7 Nested Lists and Matrices

A nested list is a list containing other lists. A common use is representing rows and columns.

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

print(matrix[0])    # First row
print(matrix[1][2]) # Row 1, column 2
print(matrix[2][0]) # Row 2, column 0

# Output:
# [1, 2, 3]
# 6
# 7
```

For `matrix[row][column]`, the first index selects the inner list/row and the second selects the element/column.

2.8 List Comprehension

List comprehension is a concise way to create a new list from an iterable. It is useful when the logic is simple and readable.

```
# Normal loop
squares = []
```

```

for number in range(1, 6):
    squares.append(number ** 2)

print(squares)
# [1, 4, 9, 16, 25]

# Same operation using list comprehension
squares = [number ** 2 for number in range(1, 6)]

print(squares)
# [1, 4, 9, 16, 25]

```

Read it as: For every number in the range, calculate number ** 2 and put the result into the new list.

List comprehension with a condition

```

numbers = [1, 2, 3, 4, 5, 18, 91]

filtered = [
    number
    for number in numbers
    if number != 18 and number != 91
]

print(filtered)
# Output:
# [1, 2, 3, 4, 5]

```

Extracting the first column of a matrix

```

matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

first_column = [row[0] for row in matrix]

print(first_column)
# Output:
# [1, 4, 7]

```

Each row is one inner list. row[0] takes the first value from every row.

2.9 Real-World Uses of Lists

Use a list when the collection can change and order matters. Examples include a shopping cart, transaction history, model predictions, file names, student marks, API results, or a sequence of records.

3. TUPLE

3.1 What is a Tuple?

A **tuple** is an ordered collection similar to a list, but it is **immutable**. Once created, its elements cannot be directly replaced, added, or removed.

```

coordinates = (10, 20)

print(coordinates)

```

```
print(coordinates[0])

# Output:
# (10, 20)
# 10
```

3.2 Why Use a Tuple?

Use a tuple when the data represents a fixed record that should not accidentally change. Examples include coordinates, RGB values, fixed configuration values, or multiple values returned by a function.

3.3 Immutability

```
values = (1, 2, 3)

# This would raise TypeError:
# values[1] = 100

# Tuple elements cannot be directly reassigned.
```

Immutability does not mean a tuple can never contain a mutable object. For example, a tuple can contain a list, and that inner list may still be changed. The tuple's own references cannot be reassigned.

3.4 Modifying Tuple Data Indirectly

```
numbers = (1, 2, 3)

# Convert tuple to list
temp = list(numbers)

# Modify the list
temp.append(4)

# Convert back to tuple
numbers = tuple(temp)

print(numbers)
# Output:
# (1, 2, 3, 4)
```

This does not modify the original tuple in place. It creates a list, modifies that list, and creates a new tuple.

3.5 Tuple Methods

```
values = (10, 20, 20, 30)

print(values.count(20))
print(values.index(30))

# Output:
# 2
# 3
```

3.6 List vs Tuple

Feature	List	Tuple
Syntax	[1, 2, 3]	(1, 2, 3)

Feature	List	Tuple
Mutable	Yes	No
Add/remove elements	Yes	Not directly
Typical use	Changing data	Fixed data
Methods	Many	Few

4. SET

4.1 What is a Set?

A **set** is a mutable collection of **unique** elements. Sets are useful for removing duplicates, membership testing, and mathematical set operations.

```
values = {1, 2, 2, 3, 3, 3}

print(values)
# Output contains only:
# {1, 2, 3}
# Exact display order should not be relied upon.
```

Important correction: Sets are not sequence containers. They do not support normal indexing such as `my_set[0]` and do not support slicing.

4.2 Creating an Empty Set

```
empty_set = set()
empty_dict = {}

print(type(empty_set))
print(type(empty_dict))

# Output:
# <class 'set'>
# <class 'dict'>
```

This is a common interview question: `{}` creates an empty dictionary, not an empty set.

4.3 add()

```
numbers = {1, 2}

numbers.add(3)
numbers.add(3)

print(numbers)
# Contains:
# {1, 2, 3}
```

Adding the same value again does not create a duplicate.

4.4 remove() vs discard()

```
numbers = {1, 2, 3}
```

```

numbers.remove(2)
print(numbers)

# remove() raises KeyError if the value does not exist.
# numbers.remove(100)

numbers = {1, 2, 3}

# No error even though 100 is absent
numbers.discard(100)

print(numbers)
# Contains:
# {1, 2, 3}

```

Use `discard()` when the element may or may not exist and you do not want a missing value to raise an error.

4.5 pop() — Important Correction

`set.pop()` removes and returns an **arbitrary element**. Do not describe it as FIFO or LIFO and do not write logic that depends on a specific element being removed.

```

numbers = {10, 20, 30}

removed = numbers.pop()

print(removed)
print(numbers)

# One element is removed.
# Do not rely on which specific element is selected.

```

The class transcript described `set pop()` as first-in-first-out. That is not a safe Python rule. Sets do not provide queue ordering.

4.6 clear()

```

numbers = {1, 2, 3}

numbers.clear()

print(numbers)
# Output:
# set()

```

4.7 Removing Duplicates from a List

```

numbers = [1, 2, 2, 3, 3, 4]

unique_numbers = set(numbers)

print(unique_numbers)
# Contains:
# {1, 2, 3, 4}

```

If original order must be preserved, converting directly to a set may not be the correct approach. In modern Python, `list(dict.fromkeys(numbers))` is a simple order-preserving deduplication technique.

4.8 Set Operations


```

a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)  # Union: all unique elements
print(a & b)  # Intersection: common elements
print(a - b)  # Difference: in a but not b
print(a ^ b)  # Symmetric difference: in either, but not both

# Typical results:
# {1, 2, 3, 4, 5, 6}
# {3, 4}
# {1, 2}
# {1, 2, 5, 6}

```

4.9 Real-World Uses of Sets

Examples include finding unique customer IDs, removing duplicate categories, comparing two user groups, finding common skills between candidates, and checking membership efficiently.

Data-modeling caution: A set can remove duplicate values from a collection, but it does not enforce business-level uniqueness in a database. Unique email IDs or employee IDs should normally be enforced using database constraints such as `UNIQUE` or `PRIMARY KEY`.

5. DICTIONARY

5.1 What is a Dictionary?

A **dictionary** stores data as **key-value pairs**. Instead of accessing a value by a numeric position, you normally access it using a meaningful key.

```

student = {
    "name": "Udayasri",
    "course": "Data Science",
    "score": 95
}

print(student["name"])
print(student["score"])

# Output:
# Udayasri
# 95

```

Easy idea: A list asks, 'What is at position 0?' A dictionary asks, 'What is the value for the key name?'

5.2 Dictionary Properties

Dictionaries are mutable. Keys must be unique and hashable. Values can be repeated and can contain many types of objects, including lists and other dictionaries. Modern Python dictionaries preserve insertion order, but access is conceptually based on keys rather than numeric indexing.

5.3 Creating and Updating Values

```

person = {}

# Add new key-value pairs
person["name"] = "Alex"
person["age"] = 25

```

```
# Update an existing value
person["age"] = 26

print(person)
# Output:
# {'name': 'Alex', 'age': 26}
```

5.4 Accessing Nested Data

```
data = {
    "student": {
        "name": "Alex",
        "skills": ["Python", "SQL", "Machine Learning"]
    }
}

print(data["student"]["name"])
print(data["student"]["skills"][0])

# Output:
# Alex
# Python
```

First, `data['student']` gets the inner dictionary. Then `['skills']` gets the list. Finally, `[0]` gets the first item from that list.

5.5 get() vs Square Brackets

```
student = {"name": "Alex"}

print(student["name"])
print(student.get("age"))
print(student.get("age", "Not available"))

# Output:
# Alex
# None
# Not available
```

`dictionary[key]` raises `KeyError` if the key is missing. `get()` can safely return `None` or a default value.

5.6 keys(), values(), and items()

```
student = {
    "name": "Alex",
    "age": 25,
    "course": "Python"
}

print(student.keys())
print(student.values())
print(student.items())
```

`keys()` provides the keys, `values()` provides the values, and `items()` provides key-value pairs. These return dictionary view objects, not ordinary lists, although they can be converted to lists if required.

5.7 Iterating Through a Dictionary

```
student = {
    "name": "Alex",
    "age": 25
}

for key, value in student.items():
    print(key, ":", value)

# Output:
# name : Alex
# age : 25
```

5.8 Dictionary Comprehension

Dictionary comprehension creates a dictionary using a concise loop expression.

```
squares = {
    number: number ** 2
    for number in range(1, 6)
}

print(squares)

# Output:
# {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

The part before the for defines key: value. Here, the number is the key and its square is the value.

5.9 Real-World Uses of Dictionaries

Dictionaries are heavily used for JSON-like API data, configuration settings, user profiles, feature metadata, model metrics, grouped results, and structured records.

6. Mutability vs Immutability

Mutable means the object can be changed after creation. **Immutable** means the object itself cannot be changed after creation.

Type	Mutable?	Example
String	No	Text values
List	Yes	Changing collection
Tuple	No	Fixed collection
Set	Yes	Changing unique collection
Dictionary	Yes	Changing key-value mapping

```
# List: mutable
items = [1, 2, 3]
items[0] = 100
print(items)
# [100, 2, 3]

# Tuple: immutable
fixed = (1, 2, 3)
```

```
# fixed[0] = 100 # TypeError
```

7. Choosing the Correct Container

Requirement	Best choice	Reason
Ordered collection that changes	List	Mutable and indexed
Fixed ordered record	Tuple	Immutable
Only unique values	Set	Automatically avoids duplicates
Named/labeled values	Dictionary	Key-value access
Fast membership checks in unique data	Set	Designed for hash-based membership
JSON/API-style structured data	Dictionary + Lists	Natural nested structure

8. Common Mistakes to Avoid

1. **Saying lists are immutable:** Lists are mutable.
2. **Using set indexing:** `my_set[0]` is invalid.
3. **Assuming `set.pop()` is FIFO:** It removes an arbitrary element.
4. **Confusing `remove()` and `pop()`:** `remove(value)` removes by value; `pop(index)` removes by index and returns the removed item.
5. **Confusing `append()` and `extend()`:** `Append` adds one object; `extend` adds elements from an iterable.
6. **Using `{}` for an empty set:** It creates a dictionary. Use `set()`.
7. **Assuming dictionary keys can repeat:** Assigning the same key again replaces its old value.
8. **Using complicated comprehensions:** A one-line expression is not automatically better. If the logic becomes hard to read, use a normal loop.

9. Quick Interview Questions

Q1. What is the difference between a list and a tuple?

A list is mutable, while a tuple is immutable. Both are ordered and can contain duplicate values.

Q2. What is the difference between `append()` and `extend()`?

`append()` adds one object to the list. `extend()` adds each element from an iterable.

Q3. What is list comprehension?

It is a concise syntax for creating a new list from an iterable, optionally with conditions.

Q4. Why use a set?

Use a set for unique elements, membership testing, and operations such as union and intersection.

Q5. Can we access a set using an index?

No. Sets do not support positional indexing.

Q6. What is a dictionary?

A mutable mapping that stores values using unique keys.

Q7. What is the difference between `dictionary[key]` and `dictionary.get(key)`?

Square-bracket access raises `KeyError` for a missing key; `get()` can safely return `None` or a default value.

Q8. What does mutable mean?

It means the object can be changed after it is created.

10. Final Revision Cheat Sheet

```
# LIST
items = [1, 2, 3]
items.append(4)           # Add one item
items.extend([5, 6])      # Add multiple elements
items.insert(0, 100)      # Insert at index
removed = items.pop()     # Remove and return item
items.remove(2)           # Remove by value
items.sort()              # Sort in place
items.reverse()           # Reverse in place

# TUPLE
fixed = (1, 2, 3)
# Ordered and immutable

# SET
unique = {1, 2, 3}
unique.add(4)
unique.discard(100)       # Safe if missing
# No normal indexing or slicing

# DICTIONARY
person = {"name": "Alex", "age": 25}
person["age"] = 26
name = person.get("name")

# LIST COMPREHENSION
squares = [x ** 2 for x in range(5)]

# DICTIONARY COMPREHENSION
square_map = {x: x ** 2 for x in range(5)}
```

11. Recommended Practice

Do not memorize methods without using them. Create one small program that combines all four containers. Example: store students as dictionaries, keep multiple student dictionaries in a list, store fixed coordinates in tuples, and use a set to collect unique skills. This gives you a realistic understanding of why each container exists.

End of Notes

Use this document as a training reference: understand the container's purpose first, then practice the syntax and methods. The most important skill is not knowing every method from memory; it is choosing the correct data structure for the problem.